Shotgun Metagenomics to MetaPhlAn 3 Abundance Table: Replicable Runbook

Oct 19, 2026 · @Milan Agrawal

Purpose and how to reuse it

This runbook turns public shotgun-metagenomic runs from NCBI SRA into one MetaPhlAn 3 species-abundance table on a WSL2 laptop, using about 9 GB of working disk at any moment. It was run on the 156-run PD cohort (SRR15040898 to SRR15041053). To reuse it for a new project, change only the parameters below and follow the phases in order.

Parameter	Value in this run	What to change for a new project
Accession list	`sra_list.txt`, 156 run IDs	Export the new project's run list from SRA Run Selector
Read layout	Paired-end	Check the LibraryLayout column of the run table; single-end needs the script edits in Phase 4
Host to remove	Human, GRCh38	Use a bowtie2 index of the host genome (mouse, pig, none for environmental samples)
Profiler	MetaPhlAn 3.0.14, database `mpa_v30_CHOCOPhlAn_201901`	Keep if your training data used MetaPhlAn 3 clades; do not mix with MetaPhlAn 4
Threads	4	Use 2 if the laptop overheats
Working folder	`~/pd_pipeline`	Any folder inside the WSL filesystem, not `/mnt/c`

Pipeline overview and tool choices

Each sample passes through five stages one at a time, and each stage deletes its input once the next stage succeeds. A sample takes about 45 minutes, so 156 samples need roughly 5 days of wall-clock time.

1. Download the `.sra` file with `prefetch` (retried up to 3 times).
2. Convert to paired FASTQ with `fasterq-dump`.
3. Trim adapters and low-quality bases with `fastp`.
4. Remove host reads with `bowtie2` against GRCh38, keeping only unaligned pairs.
5. Profile the clean reads with MetaPhlAn 3 and save one profile per sample.

After all samples finish, `merge_metaphlan_tables.py` joins the profiles into one table.

Why MetaPhlAn 3, not 4. The training data (`Source_Data_24Oct2022.xlsx`) uses MetaPhlAn 3 clades such as `k__Bacteria|...|s__Species`. MetaPhlAn 4 reports SGB-based clades, would break the feature match, and tries to download a database of about 40 GB. MetaPhlAn 3 needs about 0.5 GB for its database.

Two conda environments.

Environment	Contains	Used for
`gutmind-tools`	sra-tools 3.4.1, fastp 1.3.7, bowtie2	Stages 1 to 4 and running the script
`mpa3`	Python 3.7, MetaPhlAn 3.0.14	Stage 5 and the final merge

The script calls MetaPhlAn 3 by its full path, so `gutmind-tools` can stay active the whole time.

Phase 0 and 1: WSL, environments, folders and accession list

Do this once per machine, then once per project for the accession list.

0.1 Start WSL and stay in the Linux filesystem.

```
wsl
cd ~
```

If WSL closes immediately, run `wsl --update` in PowerShell as administrator, then `wsl --shutdown` and start again. Never work under `/mnt/c`; it is slow and counts against Windows disk.

0.2 Raise the WSL memory cap (recommended). Create `C:\Users\<you>\.wslconfig` with the lines below, then run `wsl --shutdown`.

```
[wsl2]
memory=8GB
```

0.3 Create the two environments.

```
conda activate base
conda create -n gutmind-tools -c bioconda -c conda-forge sra-tools fastp
bowtie2 -y
conda create -n mpa3 -c bioconda -c conda-forge python=3.7 metaphlan=3.0.14 -
y
~/miniconda3/envs/mpa3/bin/metaphlan --version     # must print 3.0.14
```

If the solver is slow, install mamba (`conda install -n base -c conda-forge mamba -y`) and use `mamba create` instead.

0.4 Make the folder layout.

```
mkdir -p
~/pd_pipeline/{sra_tmp,trimmed,host_removed,profiles,logs,db/host_grch38,db/m
pa_v30}
cd ~/pd_pipeline
```

1.1 Get the accession list. On the SRA Run Selector page for the new BioProject, select all runs and download the Accession List. Also download the run table (the `SraRunTable.csv`) because it holds the sample metadata and the library layout.

1.2 Copy it into WSL and clean it.

```
cp /mnt/c/Users/<you>/Downloads/SRR_Acc_List.txt ~/pd_pipeline/sra_list.txt
sed -i 's/\r$//' sra_list.txt          # strip Windows line endings
wc -l sra_list.txt                      # must equal your sample count
head -3 sra_list.txt
```

1.3 Confirm paired-end. Open the run table and check that `LibraryLayout` is `PAIRED` and `Platform` is Illumina for every row. Mixed or single-end projects need the Phase 4 edits.

Phase 2: one-time databases

Build both databases once; every later project reuses them.

2.1 MetaPhlAn 3 database (about 0.4 GB download). The legacy `mpa_v30_CHOCOPhlAn_201901` files are archived on Zenodo, because BioBakery's own server no longer hosts them (using `--index` against it returns HTTP 404). The archive contains only the marker `.pkl` and the `.fna.bz2` sequences. It has no bowtie2 index, so you build one.

```
cd ~/pd_pipeline/db/mpa_v30
wget https://zenodo.org/records/3957592/files/mpa_v30_CHOCOPhlAn_201901.tar
wget https://zenodo.org/records/3957592/files/mpa_v30_CHOCOPhlAn_201901.md5
md5sum mpa_v30_CHOCOPhlAn_201901.tar        # compare with the .md5 file
tar -xvf mpa_v30_CHOCOPhlAn_201901.tar      # gives .pkl and .fna.bz2 only
bunzip2 -k mpa_v30_CHOCOPhlAn_201901.fna.bz2
bowtie2-build --threads 4 --large-index mpa_v30_CHOCOPhlAn_201901.fna
mpa_v30_CHOCOPhlAn_201901
ls -la                                       # expect .1.bt2l .2.bt2l .3.bt2l
.4.bt2l .rev.1.bt2l .rev.2.bt2l
rm -f mpa_v30_CHOCOPhlAn_201901.fna mpa_v30_CHOCOPhlAn_201901.tar
cd ~/pd_pipeline
```

The build takes 15 to 30 minutes. Do not close the terminal during it; a partial index causes confusing errors later. The run used the large-index format (`.bt2l`) and MetaPhlAn 3 accepted it. If your copy complains that the index is missing, rebuild without `--large-index` to get standard `.bt2` files.

2.2 Human host index (about 4 GB unpacked). Use a pre-built index instead of building GRCh38 yourself.

```
cd ~/pd_pipeline/db/host_grch38
wget https://genome-idx.s3.amazonaws.com/bt/GRCh38_noalt_as.zip
unzip GRCh38_noalt_as.zip && rm GRCh38_noalt_as.zip
ls                                           # note the shared prefix of the
.bt2 files
cd ~/pd_pipeline
```

The prefix printed by `ls` (here `GRCh38_noalt_as`) is what bowtie2's `-x` option needs. For a non-human host, download that organism's pre-built bowtie2 index instead and change `HOST_IDX` in the Phase 4 script.

Phase 3: test one sample by hand

Run the first accession manually, stage by stage, and do not start the full loop until this prints real species. A one-sample test would have exposed the errors that cost four days in the first attempt.

```
conda activate gutmind-tools
cd ~/pd_pipeline
SRR=$(head -1 sra_list.txt)

prefetch $SRR -O sra_tmp/
fasterq-dump sra_tmp/$SRR/$SRR.sra -O sra_tmp/ --split-files -e 4
ls -lh sra_tmp/                              # expect ${SRR}_1.fastq and
${SRR}_2.fastq

fastp -i sra_tmp/${SRR}_1.fastq -I sra_tmp/${SRR}_2.fastq \
      -o trimmed/${SRR}_1.trim.fastq -O trimmed/${SRR}_2.trim.fastq \
      -j logs/${SRR}_fastp.json -h logs/${SRR}_fastp.html -w 4

bowtie2 -x db/host_grch38/GRCh38_noalt_as \
        -1 trimmed/${SRR}_1.trim.fastq -2 trimmed/${SRR}_2.trim.fastq \
        --un-conc host_removed/${SRR}_clean_%.fastq -p 4 -S /dev/null

~/miniconda3/envs/mpa3/bin/metaphlan \
    host_removed/${SRR}_clean_1.fastq,host_removed/${SRR}_clean_2.fastq \
    --input_type fastq --bowtie2db db/mpa_v30 --index
mpa_v30_CHOCOPhlAn_201901 \
    --nproc 4 --bowtie2out logs/${SRR}.bowtie2.bz2 -o
profiles/${SRR}_profile.txt

grep -v '^#' profiles/${SRR}_profile.txt | head
grep -c 's__' profiles/${SRR}_profile.txt    # species rows
```

What a good result looks like:

- fastp keeps more than 95% of reads.
- bowtie2 reports an overall alignment rate to the host of a few percent or less for stool (0.2% to 0.5% here). A high rate means the wrong host index or a non-stool sample.
- The profile begins `#mpa_v30_CHOCOPhlAn_201901` and lists rows such as `k__Bacteria|p__Firmicutes` with NCBI tax IDs and relative abundances.
- The species count is in the dozens.

Then clear the test files so the loop starts clean: `rm -rf sra_tmp/* trimmed/* host_removed/*` and `rm -f logs/${SRR}.bowtie2.bz2`. Keep the profile; the loop skips samples that already have one.

Phase 4: the full automated run

The script below stops a sample at its first failed step, deletes that sample's leftovers, logs the failure and moves on. This is what prevents the two earlier disasters: raw files deleted after a crash, and failed samples filling the disk. Edit only the parameter block at the top for a new project.

```
cat > ~/pd_pipeline/run_pipeline.sh << 'EOF'
#!/bin/bash
# ---- parameters: edit for a new project ----
WORKDIR=~/pd_pipeline
LIST=sra_list.txt
THREADS=4
HOST_IDX=db/host_grch38/GRCh38_noalt_as
MPA3=~/miniconda3/envs/mpa3/bin
MPA_DB=db/mpa_v30
MPA_INDEX=mpa_v30_CHOCOPhlAn_201901
# ---------------------------------------------
cd "$WORKDIR"
mkdir -p sra_tmp trimmed host_removed profiles logs

while read -r SRR; do
  [ -z "$SRR" ] && continue
  [ -s "profiles/${SRR}_profile.txt" ] && continue      # already done
  echo "=== $SRR $(date) ===" | tee -a logs/run.log

  fail() { echo "$SRR FAILED at $1 ($(date))" | tee -a logs/failed.log
           rm -rf sra_tmp/${SRR}* trimmed/${SRR}_* host_removed/${SRR}_* ; }

  # 1. download, up to 3 tries
  ok=0
  for try in 1 2 3; do
    rm -rf sra_tmp/${SRR}
    prefetch "$SRR" -O sra_tmp/ >> logs/run.log 2>&1 && [ -s
sra_tmp/$SRR/$SRR.sra ] && { ok=1; break; }
    sleep 30
  done
```

```
  [ $ok -eq 1 ] || { fail download; continue; }

  # 2. convert to FASTQ
  fasterq-dump sra_tmp/$SRR/$SRR.sra -O sra_tmp/ --split-files -e $THREADS >>
logs/run.log 2>&1
  rm -rf sra_tmp/${SRR}
  { [ -s sra_tmp/${SRR}_1.fastq ] && [ -s sra_tmp/${SRR}_2.fastq ]; } || {
fail fasterq-dump; continue; }

  # 3. trim
  fastp -i sra_tmp/${SRR}_1.fastq -I sra_tmp/${SRR}_2.fastq \
        -o trimmed/${SRR}_1.trim.fastq -O trimmed/${SRR}_2.trim.fastq \
        -j logs/${SRR}_fastp.json -h logs/${SRR}_fastp.html -w $THREADS >>
logs/run.log 2>&1 \
        || { fail fastp; continue; }
  rm -f sra_tmp/${SRR}_*.fastq

  # 4. remove host reads
  bowtie2 -x $HOST_IDX \
          -1 trimmed/${SRR}_1.trim.fastq -2 trimmed/${SRR}_2.trim.fastq \
          --un-conc host_removed/${SRR}_clean_%.fastq \
          -p $THREADS -S /dev/null >> logs/run.log 2>&1 \
          || { fail bowtie2; continue; }
  rm -f trimmed/${SRR}_*
  [ -s host_removed/${SRR}_clean_1.fastq ] || { fail
empty_after_host_removal; continue; }

  # 5. MetaPhlAn 3
  rm -f logs/${SRR}.bowtie2.bz2
  PATH=$MPA3:$PATH $MPA3/metaphlan \
      host_removed/${SRR}_clean_1.fastq,host_removed/${SRR}_clean_2.fastq \
      --input_type fastq --bowtie2db $MPA_DB --index $MPA_INDEX \
      --nproc $THREADS --bowtie2out logs/${SRR}.bowtie2.bz2 \
      -o profiles/${SRR}_profile.txt >> logs/run.log 2>&1

  if [ -s "profiles/${SRR}_profile.txt" ] && grep -q "s__"
"profiles/${SRR}_profile.txt"; then
    echo -e "${SRR}\t$(( $(wc -l < host_removed/${SRR}_clean_1.fastq) / 4
))\t$(grep -c 's__' profiles/${SRR}_profile.txt)\t$(date +%F_%T)" >>
logs/qc.tsv
    rm -f host_removed/${SRR}_clean_*.fastq
    echo "$SRR OK" | tee -a logs/run.log
  else
```

```
    rm -f "profiles/${SRR}_profile.txt"; fail metaphlan
  fi
done < "$LIST"
echo "ALL DONE $(date)" | tee -a logs/run.log
EOF

chmod +x ~/pd_pipeline/run_pipeline.sh
bash -n ~/pd_pipeline/run_pipeline.sh && echo "syntax OK"
```

The `logs/qc.tsv` line (clean read pairs and species rows per sample) was added after the run began; it is included here so the next project gets the QC record from the first sample.

Start it in the background, so closing the terminal tab does not stop it:

```
conda activate gutmind-tools
cd ~/pd_pipeline
nohup ./run_pipeline.sh > logs/nohup.out 2>&1 &
tail -f logs/run.log        # Ctrl+C stops watching only; the job keeps
running
```

Edits for other layouts. Single-end runs: remove the `-I`, `-O` and `_2` arguments in steps 2 to 5, use `--split-files` output `${SRR}.fastq`, and pass one file to MetaPhlAn. A different host: change `HOST_IDX`. Never edit the script while it runs; stop it, edit, restart.

Resume after any interruption. Finished samples are skipped automatically, so recovery is three commands:

```
for f in profiles/*_profile.txt; do grep -q "s__" "$f" || { echo "bad: $f";
rm -f "$f"; }; done
rm -rf sra_tmp/* trimmed/* host_removed/*
nohup ./run_pipeline.sh > logs/nohup.out 2>&1 &
```

Phase 5: monitoring and catching silent failures

An exit code is not proof of a good result, so check three things: the job is alive, each sample looks plausible, and the counts add up.

Is it still running? A closed WSL window, a Windows sleep or a power loss ends the job without any message.

```
pgrep -af run_pipeline.sh || echo "NOT RUNNING"
tail -n 3 logs/run.log
ls -lt profiles | head -3          # newest profile; should be under about an
hour old
df -h ~                            # should stay flat, not grow by 9 GB per
sample
```

If the job runs but no profile is new for over an hour, look at `ls -la sra_tmp/`. A `.tmp` file that is not growing means a stalled download; stop the job and restart it.

Is each result plausible? With `logs/qc.tsv` in place, list the weak samples.

```
column -t logs/qc.tsv | tail
awk '$2<1000000 || $3<10' logs/qc.tsv      # fewer than 1M clean pairs or
under 10 species
grep "overall alignment rate" logs/run.log | sort | uniq -c | sort -rn | head
```

Red flag	Likely meaning
Under about 1 million clean read pairs	Sample too shallow to trust
Under about 10 species rows	Almost nothing mapped, wrong sample type or bad reads
Host alignment rate far above a few percent for stool	Wrong host index, or not a stool sample
Identical numbers in different samples	A file is being reused

Do the counts add up? When the run ends, done plus failed must equal total.

```
echo "done:   $(ls profiles/*_profile.txt 2>/dev/null | wc -l)"
echo "failed: $(cut -d' ' -f1 logs/failed.log 2>/dev/null | sort -u | wc -l)"
echo "total:  $(wc -l < sra_list.txt)"
comm -23 <(sort sra_list.txt) <(ls profiles | sed 's/_profile.txt//' | sort)
# samples with no profile
```

If any sample failed, run the same `nohup` command again; the script retries only samples without a profile. Read `logs/failed.log` first, because it names the step that failed. Archive an old `failed.log` (`mv logs/failed.log logs/failed_old.log`) before a restart so every remaining line is a real failure.

Watch the first full sample. Before leaving the job unattended, watch sample 1 finish in a second terminal and confirm its intermediate files disappear and disk use stays flat.

Phase 6: merge, species table and export

The final table is one MetaPhlAn 3 merge plus a filter to species rows. Run it only after the counts in Phase 5 add up.

```
conda activate mpa3
cd ~/pd_pipeline
python ~/miniconda3/envs/mpa3/bin/merge_metaphlan_tables.py
profiles/*_profile.txt > final_abundance_table.txt

# species rows only (drops genus, family and strain rows)
grep -E "^clade_name|^ID|s__" final_abundance_table.txt | grep -v "t__" >
final_species_abundance_table.txt

head -1 final_species_abundance_table.txt | tr '\t' '\n' | wc -l     #
samples + 1 label column
wc -l final_species_abundance_table.txt                                 #
species rows + header
```

The column count must equal the number of profiles plus one (157 for 156 samples). Each sample's species rows should add to about 100, because MetaPhlAn 3 reports percentages.

Copy the results to Windows.

```
mkdir -p /mnt/c/Users/<you>/Documents/pd_results
cp final_abundance_table.txt final_species_abundance_table.txt
/mnt/c/Users/<you>/Documents/pd_results/
explorer.exe "$(wslpath -w /mnt/c/Users/<you>/Documents/pd_results)"
```

If the folder does not appear in Documents, Windows is probably showing `C:\Users\<you>\OneDrive\Documents` instead. Copy to that path, or browse `\\wsl.localhost\Ubuntu\home\<you>\pd_pipeline`.

Using the table for machine learning.

1. Transpose it so rows are samples and columns are species.
2. Join to the `SraRunTable.csv` metadata on the `Run` column to attach disease status and other labels.

3. Align the species columns to the training data's feature list: add any missing species as 0 and drop species the model never saw.
4. Keep the abundance scale (percent) the same as the training data, or convert both to fractions.
5. Keep the per-sample `profiles/` folder as the backup of record.

Troubleshooting: every problem met in this run

These are the failures that occurred, in the order they appeared, with the fix that worked.

Symptom	Cause	Fix
`unrecognized arguments: --bowtie2out`	MetaPhlAn 4 renamed the option	MetaPhlAn 4 uses `--mapout`; MetaPhlAn 3 uses `--bowtie2out`. Use MetaPhlAn 3 here
Four days of runs produced no profiles and no raw files	The old script ran its cleanup line even after MetaPhlAn crashed	Delete files only after a non-empty profile exists, and stop each sample at its first failure (Phase 4 script)
MetaPhlAn starts downloading a database of about 40 GB	MetaPhlAn 4 default behaviour	Stop it; use the MetaPhlAn 3 database from Phase 2
`HTTP Error 404` with `--index mpa_v30_CHOCOPhlAn_201901`	BioBakery archived the legacy database	Download it from Zenodo (Phase 2)
Disk fills at about 26 GB per failed sample	Raw, trimmed and host-removed FASTQ all kept until the end	Delete each stage's input once the next stage finishes
`--mapout needs to be specified when multiple FASTQ files are provided`	Paired files passed as `file1,file2`	Always pass the output-map option (`--bowtie2out` in MetaPhlAn 3)
No `.bt2` files after extracting the tar	The Zenodo archive has only the `.pkl` and `.fna.bz2`	Build the index with `bowtie2-build` (Phase 2)

Symptom	Cause	Fix
`IndexError: list index out of range` in `build_tree`	MetaPhlAn 4 code reading a MetaPhlAn 3 `.pkl`	Run MetaPhlAn 3.0.14 from the separate `mpa3` environment
`Input file not found` for the wrong SRR	A stale `$SRR` shell variable	Set `SRR=...` explicitly before testing by hand
`BowTie2 output file detected ... remove it`	A `.bowtie2.bz2` from an earlier run exists	`rm logs/${SRR}.bowtie2.bz2` (the script does this)
`transfer interrupted ... failed to download`	Network drop during `prefetch`	The script retries 3 times; rerun the script for any that still fail
Laptop shut down overnight	Most likely overheating under 4 busy threads	Restart with the three resume commands in Phase 4; lower `THREADS` to 2; check Event Viewer for Kernel-Power ID 41
`pd_results` folder missing from Documents	OneDrive redirects Documents	Open it with `explorer.exe` or copy to the OneDrive path
`Failed to launch x86-64-v3 version`	Tool falls back to its default build	Harmless warning; ignore
`WARNING: ... clades that represent multiple species merged`	Normal MetaPhlAn 3 message	Harmless; an extra column lists the merged species

Power and sleep. Set Windows to never sleep while plugged in, keep the laptop on a hard surface with the vents clear, and run `pgrep -af run_pipeline.sh` after any restart, because `nohup` survives a closed terminal but not a power loss.

Results of this run and replication checklist

All 156 samples produced a valid MetaPhlAn 3 profile, and the final run logged no failures.

Measure	Value
Samples with a profile	156 of 156

Measure	Value
Entries in `failed.log` at the end	0
Lowest species count in any sample	33 (SRR15040988)
Next-lowest species counts	44, 48, 54
Clean read pairs seen in spot checks	about 10 to 14 million per sample
Host alignment rate in spot checks	0.2% to 0.5%
Time per sample	about 45 minutes
Disk in use during the run	about 24 GB, flat
Interruption	One overnight laptop shutdown; resumed with no sample lost

Replication checklist for a new project ID.

- [ ] WSL running; `gutmind-tools` and `mpa3` environments exist; `metaphlan --version` in `mpa3` prints 3.0.14
- [ ] Both databases built and the six index files present (Phase 2)
- [ ] New accession list saved as `sra_list.txt`, Windows line endings removed, count matches the run table
- [ ] Run table shows paired-end Illumina for every sample, or the script edits for single-end are made
- [ ] Host index matches the organism sampled
- [ ] One sample tested by hand and the profile shows dozens of species (Phase 3)
- [ ] Parameter block at the top of `run_pipeline.sh` edited; `bash -n` reports no errors
- [ ] Windows set to never sleep while plugged in; laptop ventilated
- [ ] Job started with `nohup`; first sample watched to completion
- [ ] At the end: done plus failed equals total, `qc.tsv` has no red flags
- [ ] Merged table has samples plus one columns; results copied out of WSL and `profiles/` backed up